

Study Note: The AI and Data Science Periodic Tables: A Unified Taxonomic Blueprint for Enterprise Systems

Muhammad Sukri Bin Ramli

Asia School of Business, Kuala Lumpur, Malaysia

Email: m.binramli@sloan.mit.edu

August 13, 2026

Abstract

This architecture reference note establishes a formalized, single-column taxonomic mapping framework that unifies classical data science workflows with modern artificial intelligence (AI) operational runtimes. Based on the technical framework introduced by Aaron Baughman and Martin Keen, this document addresses industry jargon fragmentation by organizing system capabilities into standardized, matrix-based periodic tables. By pairing the Data Science Periodic Table with the Artificial Intelligence Periodic Table, enterprise architecture teams can map complex end-to-end interactions into modular chemical-style reactions. This note deconstructs a 12-element linear Enterprise Document Question and Answering (Q&A) pipeline and outlines its transformation into a 15-element recursive, self-improving loop driven by automated drift detection, synthetic data generation, and model fine-tuning.

1 Introduction and Taxonomic Scope

The rapid growth of generative artificial intelligence and enterprise machine learning operations has created widespread terminology fragmentation across data engineering, software orchestration, and algorithmic alignment domains. While enterprise focus has aggressively shifted toward artificial intelligence capabilities, advanced artificial intelligence runtime systems remain fundamentally bound to foundational data science practices. High-dimensional vector embeddings, retrieved text chunks, and large language model outputs are mathematically meaningless without rigorous data extraction, cleansing, metadata structuring, and governance. Conversely, modern data science workflows increasingly leverage artificial intelligence models to automate data labeling, compute density distributions, and synthesize artificial training records.

To resolve this mutual dependence and eliminate domain jargon, Aaron Baughman and Martin Keen introduced a dual-table framework that organizes classical data science and artificial intelligence into structured, periodic reference guides. Analogous to chemical elements in physical chemistry, individual operational capabilities are isolated into two-letter atomic symbols organized systematically along vertical functional groups and horizontal maturity layers. This unified framework allows system architects to transition away from fragmented software descriptions and toward formal system reactions, where enterprise applications are represented as deterministic, traceable sequences of interacting elements.

2 The Visual Taxonomy Matrix

As visually detailed in Figure 1, the taxonomy divides system design into two complementary grids: the Classical Data Science Periodic Table, which governs data lifecycle, cleanliness, and validation, and the Artificial Intelligence Periodic Table, which manages model runtime, orchestration, and alignment. The Data Science Periodic Table categorizes operations across five lifecycle columns—Acquisition, Preparation, Modeling, Generation, and Evaluation—spanning

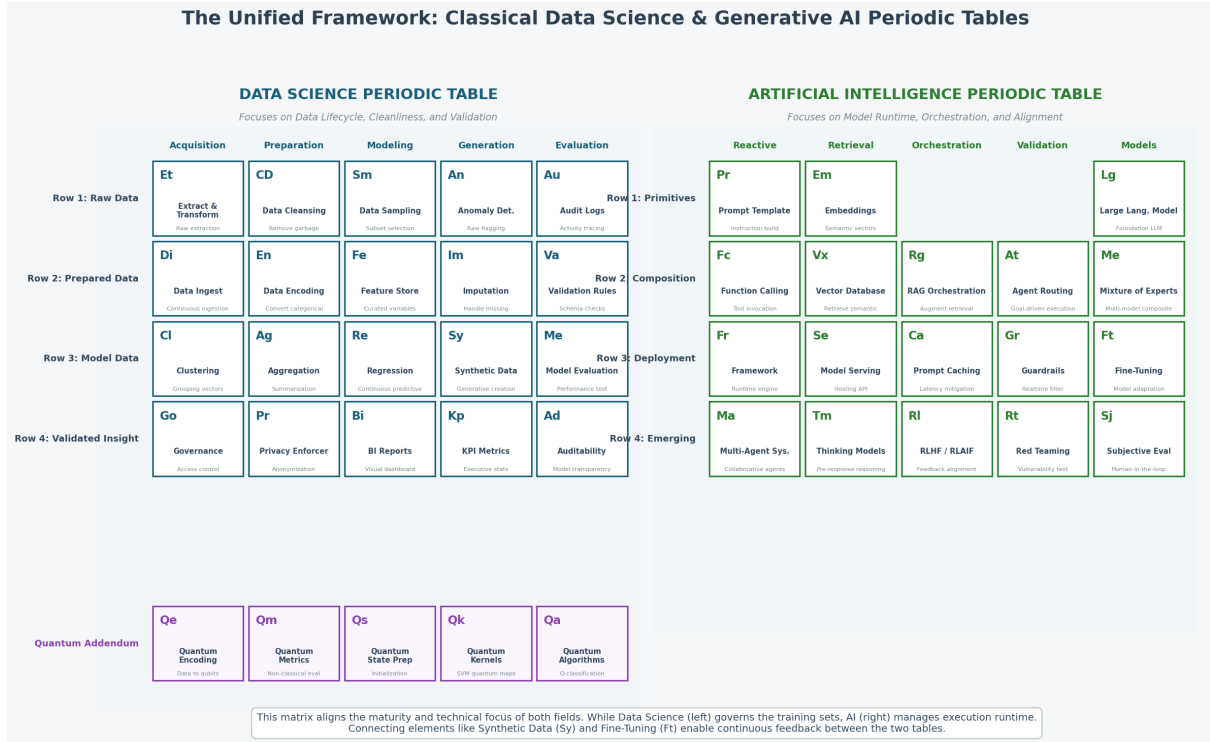


Figure 1: The Unified Framework: Classical Data Science and Generative AI Periodic Tables. Reproduced from the technical architecture framework by Aaron Baughman and Martin Keen.

four data maturity rows ranging from Raw Data to Validated Insights, alongside an extended Quantum Computing Addendum.

The Artificial Intelligence Periodic Table side of Figure 1 categorizes runtime execution across five functional groups and four deployment maturity rows. Group 1 (Reactive) isolates input-acting execution primitives such as Prompt Templates (*Pr*), Function Calling (*Fc*), Agent Frameworks (*Fr*), and Multi-Agent Systems (*Ma*). Group 2 (Retrieval) captures the memory side of artificial intelligence, incorporating Embeddings (*Em*), Vector Databases (*Vx*), Model Serving (*Se*), and Thinking Models (*Tm*). Group 3 (Orchestration) handles runtime coordination, including Retrieval-Augmented Generation (*Rg*), Prompt Caching (*Ca*), and Reinforcement Learning from Human or AI Feedback (*RI*). Group 4 (Validation) enforces operational safety through Agent Routing (*At*), Guardrails (*Gr*), and Red Teaming (*Rt*). Finally, Group 5 (Models) organizes foundational runtimes, including Large Language Models (*Lg*), Mixture of Experts (*Me*), Fine-Tuning (*Ft*), and Subjective Evaluation (*Sj*).

Vertical maturity progresses from Row 1 (Primitives), containing atomic building blocks like Prompts (*Pr*), Embeddings (*Em*), and Large Language Models (*Lg*), to Row 2 (Composition) for production routines like Function Calling (*Fc*), Row 3 (Deployment) for runtime frameworks, and Row 4 (Emerging) for multi-agent coordination.

3 Systematic Deconstruction of an Enterprise Reaction Pipeline

In this architectural model, periodic tables predict reactions, where a reaction represents an operational enterprise software system constructed by chaining atomic elements across both tables. A prime application is the Enterprise Document Question and Answering system (Document Q&A), designed to allow corporate analysts to query distributed internal policies without risk of model hallucination or unauthorized data leakage.

Building a secure Document Q&A system requires a deterministic 12-element linear pipeline

How Data Science & AI Periodic Tables Combine into a Self-Improving System

Mapping of 'Elements' from raw data preparation to AI runtime and the continuous retraining loop

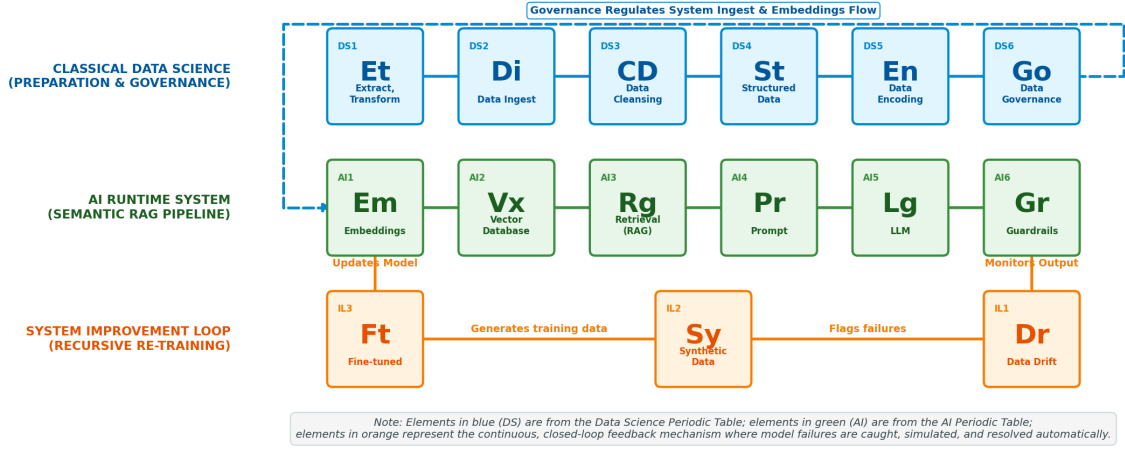


Figure 2: Mapping of elements from raw data preparation to AI runtime and the continuous retraining loop.

split evenly between classical data science prep work and active artificial intelligence runtime execution, as illustrated in the top and middle tiers of Figure 2. The data science pipeline opens with Extract, Transform, Load (*Et*), which gathers raw policy documents from scattered corporate repositories like SharePoint, Confluence, and file shares. To prevent the system from answering queries based on outdated policies, Data Ingest (*Di*) continuously syncs newly updated documents into the pipeline. Incoming text streams pass to Data Cleansing (*CD*) to strip out extraneous PDF artifacts, watermarks, headers, footers, and optical character recognition noise.

Cleaned text blocks move to Structured Data (*St*), where raw files are parsed into semantic chunks and tagged with metadata including department origin, effective date, and sensitivity tiers. These tags are transformed via Data Encoding (*En*) into standardized numerical schema formats suitable for precise metadata filtering. Before handing off data to the artificial intelligence runtime, Data Governance (*Go*) enforces strict access permissions and audit trails, ensuring that confidential document chunks remain inaccessible to unauthorized user roles.

Once data preparation completes, the artificial intelligence runtime layer converts clean text chunks into high-dimensional numerical vectors using Embeddings (*Em*). These vector representations are indexed inside a Vector Database (*Vx*) to enable fast semantic similarity searches during inference. When a financial analyst submits a query at runtime, the system triggers Retrieval-Augmented Generation (*Rg*), embedding the user's question and executing a vector search against *Vx*, constrained strictly by the user's *Go* access boundaries.

Retrieved text chunks are formatted along with the analyst's query into a structured Prompt Template (*Pr*), providing explicit grounding context that prevents foundational model hallucinations. The assembled prompt passes to the Large Language Model (*Lg*), which generates an answer cited directly from retrieved sources. Finally, output text passes through Guardrails (*Gr*), which validates citation fidelity, verifies response accuracy, and redacts any residual personally identifiable information before returning the answer to the user interface.

4 Continuous Feedback and Recursive System Improvement Loop

While the 12-element linear reaction executes stateless inference, real-world enterprise environments experience continuous semantic drift, evolving vocabulary, and complex retrieval failures. As highlighted in the bottom layer of Figure 2, turning a static linear pipeline into an adaptive, self-improving system requires adding three specialized feedback elements: Data Drift (*Dr*), Synthetic Data (*Sy*), and Fine-Tuning (*Ft*).

The continuous improvement loop operates asynchronously by monitoring real-time query executions and tracking user feedback signals. When user satisfaction metrics degrade or incoming query embeddings drift away from established baseline clusters, Data Drift (*Dr*) flags specific retrieval breakdown patterns. Instead of requiring manual engineering intervention, these flagged edge cases are forwarded to Synthetic Data (*Sy*), where generative models automatically construct synthetic question-and-answer pairs designed to model failing topics.

These synthetic Q&A pairs are packaged into specialized fine-tuning datasets and passed to Fine-Tuning (*Ft*). The *Ft* process updates the parameters of the underlying embedding model (*Em*), explicitly training vector spaces to map complex domain queries closer to relevant policy document chunks. Once re-indexed inside the Vector Database (*Vx*), the system automatically resolves previous failure modes, establishing a closed-loop feedback mechanism that continuously optimizes system accuracy without manual retraining overhead.

5 Conclusion

By establishing a unified, single-column reference architecture across classical data science and modern artificial intelligence, this study note provides enterprise system architects with a structured methodology for designing operational platforms. Mapping capabilities to explicit periodic table coordinates simplifies complex system evaluation, exposes technical debt, and provides an actionable blueprint for enterprise adoption. As demonstrated by the Document Q&A implementation, linking data governance and cleansing with generative retrieval creates robust, scalable, and self-improving enterprise software systems.

References

- [1] A. Baughman and M. Keen, “AI & Data Science Periodic Tables: How They Work Together,” IBM Technology, Technical Lightboard Lecture, 2026. Available at: https://www.youtube.com/watch?v=jAkB_qeATag.